S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

Practical No. 02

Aim: Apply the concept of production rules to solve the water jug problem. (A Water Jug Problem: You are given two jugs, a 4-gallon one and a 3-gallon one, a pump which has unlimited water which you can use to fill the jug, and the ground on which water may be poured. Neither jug has any measuring markings on it. How can you get exactly 2 gallons of water in the 4-gallon jug?)

Name of Student:

Roll No.:

Semester/Year: V/III

Academic Session: 2025-2026

Date of Performance:

Date of Submission:

AIM: Apply the concept of production rules to solve the water jug problem. (A Water Jug Problem: You are given two jugs, a 4-gallon one and a 3-gallon one, a pump which has unlimited water which you can use to fill the jug, and the ground on which water may be poured. Neither jug has any measuring markings on it. How can you get exactly 2 gallons of water in the 4-gallon jug?)

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To be able to understand the concept of Water Jug.
- To be able to acquire the puzzle solving capability.

THEORY:

The water jug problem is a classic puzzle in artificial intelligence and computer science that involves solving a problem related to filling and measuring water using two or more jugs of known capacities. This problem serves as an interesting and practical example of how AI algorithms can be applied to find solutions for real-world problems.

The basic setup of the water jug problem typically involves two or more jugs of different capacities, with the goal of measuring or obtaining a specific amount of water. Here's a common version of the problem using two jugs:

Problem Statement:

You are given two jugs with capacities A liters and B liters, where A and B are positive integers. The jugs have no markings to indicate their current level of water. You also have access to a water source and a sink. Your task is to measure exactly C liters of water, where C is a positive integer, using these jugs.

- You can perform the following operations:
- Fill a jug from the water source.
- Empty a jug into the sink.

Pour the water from one jug into the other until either the source jug is empty or the target jug is full. The objective is to find a series of operations that will result in having C liters of water in one of the jugs or determining that it is impossible to measure C liters with the given jugs and operations. In the **water jug problem in Artificial Intelligence**, we are provided with two jugs: one having the capacity to hold 3 gallons of water and the other has the capacity to hold 4 gallons of water. There is no other measuring equipment available and the jugs also do not have any kind of marking on them. So, the agent's task here is to fill the 4-gallon jug with 2 gallons of water by using only these two jugs and no other material. Initially, both our jugs are empty.

So, to solve this problem, following set of rules were proposed:

Production rules for solving the water jug problem

Here, let ***x*** denote the 4-gallon jug and ***y*** denote the 3-gallon jug.

S.No.	Initial State	Condition	Final state	Description of action taken
1.	(x,y)	If x<4	(4,y)	Fill the 4 gallon jug completely
2.	(x,y)	if y<3	(x,3)	Fill the 3 gallon jug completely
3.	(x,y)	If x>0	(x-d,y)	Pour some part from the 4 gallon jug
4.	(x,y)	If y>0	(x,y-d)	Pour some part from the 3 gallon jug
5.	(x,y)	If x>0	(0,y)	Empty the 4 gallon jug
6.	(x,y)	If y>0	(x,0)	Empty the 3 gallon jug
7.	(x,y)	If (x+y)<7	(4, y-[4-x])	Pour some water from the 3 gallon jug to fill the four gallon jug
8.	(x,y)	If (x+y)<7	(x-[3-y],y)	Pour some water from the 4 gallon jug to fill the 3 gallon jug.

9.	(x,y)	If (x+y)<4	(x+y,0)	Pour all water from 3 gallon jug to the 4 gallon jug
10.	(x,y)	if (x+y)<3	(0, x+y)	Pour all water from the 4 gallon jug to the 3 gallon jug

The listed production rules contain all the actions that could be performed by the agent in transferring the contents of jugs. But, to solve the water jug problem in a minimum number of moves, following set of rules in the given sequence should be performed:

Solution of water jug problem according to the production rules

S.No.	**4 gallon jug contents**	**3 gallon jug contents**	**Rule followed**
1.	0 gallon	0 gallon	Initial state
2.	0 gallon	3 gallons	Rule no.2
3.	3 gallons	0 gallon	Rule no. 9
4.	3 gallons	3 gallons	Rule no. 2
5.	4 gallons	2 gallons	Rule no. 7
6.	0 gallon	2 gallons	Rule no. 5

7.	2 gallons	0 gallon	Rule no. 9

On reaching the 7th attempt, we reach a state which is our goal state. Therefore, at this state, our problem is solved.

PROGRAM CODE:

1. **Approach BFS**

Here, we keep exploring **all the different valid cases of the states of water in the jug simultaneously** until and unless we reach the required target water. As provided in the problem statement, at any given state we can do either of the following operations:

- Fill a jug
- Empty a jug
- Transfer water from one jug to another **until either of them gets completely filled or empty.**

2. **Approach Memoization:**

An approach using BFS has been discussed in the previous para. In this para an approach using memoization and recursion is been discussed. At any point, there can be a total of six possibilities:

- Empty the first jug completely
- Empty the second jug completely
- Fill the first jug
- Fill the second jug
- Fill the water from the second jug into the first jug until the first jug is full or the second jug has no water left
- Fill the water from the first jug into the second jug until the second jug is full or the first jug has no water left

3. **Approach Recursion :**

Using Recursion, visit all the six possible moves one by one until one of them returns True. Since there can be repetitions of same recursive calls, hence every return value is stored using memoization to avoid calling the recursive function again and returning the stored value.

Time Complexity: O(n*m).
Space Complexity: O(n*m)**. Where n and m are the quantity of jug1 and jug2, respectively.**

ALGORITHM:

- Initialize two variables ***j1*** and ***j2*** to represent the current amount of water in each jug.
- Set a target amount of water to measure out.
- Create a list of possible actions, including filling a jug, emptying a jug, and pouring water from one jug to the other.
- Create an empty set to keep track of visited states.
- Create a stack to keep track of states to visit, and add the initial state to the stack.
- While the stack is not empty:
    - Pop the top state from the stack.
    - If this state has not been visited before, add it to the visited set.
    - Check if the state matches the target amount of water. If so, return the ***seq*** actions taken to get to this state.
    - If the state does not match the target amount, generate all possible next states from this state by applying each of the possible actions in turn.
    - If it has not been visited before, add it to the stack for each next state.
- If the stack becomes empty without finding a solution, return ***None***.

CODE :

```
# Water Jug Problem using DFS (m, n, d, goal)

def water_jug_dfs(m, n, d, goal):
    solutions = []

    def dfs(jug1, jug2, path, visited):
        if (jug1, jug2) in visited:
            return
        visited.add((jug1, jug2))
        path.append((jug1, jug2))

        # Goal reached
        if (jug1, jug2) == goal:
            solutions.append(path.copy())
            path.pop()
            return

        # All possible actions
        actions = [
            (m, jug2),  # Fill jug1
            (jug1, n),  # Fill jug2
            (0, jug2),  # Empty jug1
            (jug1, 0),  # Empty jug2
            (max(0, jug1 - (n - jug2)), min(n, jug1 + jug2)),  # jug1 → jug2
            (min(m, jug1 + jug2), max(0, jug2 - (m - jug1)))   # jug2 → jug1
        ]
```

```
        for state in actions:
            dfs(state[0], state[1], path, visited.copy())

        path.pop()

    dfs(0, 0, [], set())
    return solutions


# -------- USER INPUT --------

m = int(input("Enter capacity of Jug 1 (m): "))
n = int(input("Enter capacity of Jug 2 (n): "))
d = int(input("Enter desired amount (d): "))

goal_input = input("Enter goal state in format <x,y>: ")
goal = tuple(map(int, goal_input.strip("<>").split(",")))

# -------- VALIDATION --------

if goal[0] < 0 or goal[1] < 0 or goal[0] > m or goal[1] > n:
    print("\n Invalid goal state")
elif goal[0] != d and goal[1] != d:
    print("\n Invalid goal state (d not present)")
else:
    solutions = water_jug_dfs(m, n, d, goal)

    if not solutions:
        print("\n No solution found")
    else:
        print("\n POSSIBLE SOLUTIONS:\n")

        for i, sol in enumerate(solutions, 1):
            print(f"Solution {i} (Steps = {len(sol)-1}):")
            print(sol)
            print()

        # Best solution = minimum steps
        best_solution = min(solutions, key=len)

        print(" BEST SOLUTION (Minimum Steps):")
        print(best_solution)
        print(f"Total steps: {len(best_solution)-1}")
```

INPUT & OUTPUT:

```
Run    PR 02 code

C:\Users\GAURAV\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\GAURAV\Desktop\AI\PR 02 code.py"
Enter capacity of Jug 1 (m): 4
Enter capacity of Jug 2 (n): 3
Enter desired amount (d): 2
Enter goal state in format <x,y>: <2,0>

✅ POSSIBLE SOLUTIONS:

Solution 1 (Steps = 8):
[(0, 0), (4, 0), (4, 3), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 2 (Steps = 9):
[(0, 0), (4, 0), (1, 3), (4, 3), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 3 (Steps = 8):
[(0, 0), (4, 0), (1, 3), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 4 (Steps = 12):
[(0, 0), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (4, 3), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 5 (Steps = 13):
[(0, 0), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (4, 3), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 6 (Steps = 12):
[(0, 0), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 7 (Steps = 7):
[(0, 0), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (2, 0)]

Solution 8 (Steps = 10):
[(0, 0), (4, 0), (1, 3), (1, 0), (0, 1), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 9 (Steps = 9):
[(0, 0), (0, 3), (4, 3), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (2, 0)]

Solution 10 (Steps = 9):
[(0, 0), (0, 3), (3, 0), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (2, 0)]

Solution 11 (Steps = 11):
[(0, 0), (0, 3), (3, 0), (3, 3), (4, 3), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (2, 0)]

Solution 12 (Steps = 12):
[(0, 0), (0, 3), (3, 0), (3, 3), (4, 2), (4, 3), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (2, 0)]

Solution 13 (Steps = 6):
[(0, 0), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]

Solution 14 (Steps = 11):
[(0, 0), (0, 3), (3, 0), (3, 3), (4, 2), (4, 0), (1, 3), (1, 0), (0, 1), (4, 1), (2, 3), (2, 0)]

⭐ BEST SOLUTION (Minimum Steps):
[(0, 0), (0, 3), (3, 0), (3, 3), (4, 2), (0, 2), (2, 0)]
Total steps: 6
```

Enter Jar 1 Capacity: - 4
Enter Jar2 Capacity: - 3

O/P

Path of states of jugs followed are:

<0 , 0> // Initial State
<0 , 3> // Fill Jar 2 with water
<3 , 0> // Transfer water from Jar 2 to Jar 1
<3 , 3> // Fill Jar 2 with water
<4 , 2> // Transfer water from Jar 2 to Jar 1
<0 , 2> // Empty Jar 1 (Goal Reached)

CONCLUSION:

DISCUSSION QUESTIONS:

- Which Technique is best for water jug problem?

- Give second solution for water jug problem and write production rules.

REFERENCES:

- https://www.geeksforgeeks.org/water-jug-problem-using-bfs/
- https://www.geeksforgeeks.org/water-jug-problem-using-memoization/
- https://favtutor.com/blogs/water-jug-problem
- https://www.codingninjas.com/studio/library/solving-water-jug-problem-using-bfs
- https://www.javatpoint.com/water-jug-problem-in-python